

Learning with AI: A Systematic Review of Novice Programmer Use of Large Language Model-based Coding Assistants, Benefits, Downsides, and Challenges

Duc Manh Bui

Learning with AI: A Systematic Review of Novice Programmer Use of Large Language Model-based Coding Assistants, Benefits, Downsides, and Challenges

Duc Manh Bui

Thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Technology.
Otaniemi, 16 May 2025

Supervisor: University Lecturer Matti Raasakka
Advisor: Senior University Lecturer Arto Hellas
Academy Research Fellow Juho Leinonen

Aalto University
School of Science
Bachelor's Programme in Science and Technology

Author

Duc Manh Bui

Title

Learning with AI: A Systematic Review of Novice Programmer Use of LLM-based Coding Assistants, Benefits, Challenges, and Educational Impact

School School of Science**Degree programme** Bachelor's Programme in Science and Technology**Major** Quantum Technology**Code** SCI3103**Supervisor** University Lecturer Matti Raasakka**Advisor** Senior University Lecturer Arto Hellas, Academy Research Fellow Juho Leinonen**Level** Bachelor's thesis **Date** 16 May 2025 **Pages** 43 **Language** English**Abstract**

The recent advances in generative AI (genAI) and LLM-based coding assistants have introduced new possibilities for computing education. While many focus on potential applications of these programming assistants, others express concern about their effect on novice programmers and strive to establish the best way to integrate these tools into the classroom. This thesis performs a systematic literature review of 22 studies to examine how novice programmers use LLM-based coding assistants and what educational benefits and downsides these bring. It further investigates the limitations of the tools and challenges in using them. This thesis found that novice programmers use coding assistants mainly for asking immediate help instead of improving their skills and knowledge. This usage trend led to the main downsides of overreliance and impeded skill development. Moreover, coding assistants provide some benefits, including efficiency and an improved learning experience. Additionally, novice programmers report the challenge of coding assistants often "misunderstanding" them and the problem of inaccuracy responses by the model. From these findings, this thesis highlights several implications for both learning programming and programming educators. Finally, it suggests recommendation practices on how to best adapt and integrate LLM-based coding assistants to support novice programmers' study.

Keywords generative AI, large language model, artificial intelligence, programming assistant, programming tutors, novice programmers, student, computing education

urn <https://aaltodoc.aalto.fi>

Contents

Abstract	ii
Contents	iii
1. Introduction	1
2. Background	3
2.1 Generative Artificial Intelligence and Large Language Model	3
2.2 Large Language Model-based Coding Assistants in Programming Education	4
3. Systematic Literature Review Approach	6
3.1 Methodology and Research Questions	6
3.2 Search Strategy	7
3.3 Inclusion and Exclusion Criteria	8
3.4 Data Extraction Method	9
4. Results	11
4.1 Metadata of Reviewed Papers	11
4.2 Classification of the Literature	12
4.3 Classification of Large Language Model-based Coding Assistant	14
4.4 RQ1 What Is the Current Design of Large Language Model-based Programming Assistants?	15
4.5 RQ2 How Have Large Language Model-based Programming Assistants Been Used In The Research Literature?	18
4.6 RQ3 What Are the Educational Benefits for Novice Programmers Using Programming Assistants?	19
4.7 RQ4 What Are the Educational Downsides for Novice Programmers Using Programming Assistants?	21

4.8	RQ5 What Are the Challenges in Using Programming Assistants and Limitations of the Tool?	23
5.	Discussion	27
5.1	Summarizing Answers to Research Questions	27
5.2	Implications for Learning Programming	29
5.3	Implications for Programming Educators	31
5.4	Recommendation for Integrating Large Language Model-based Coding Assistant	32
5.5	Limitations of the Study	33
6.	Conclusion	34
A.	Appendix	42

1. Introduction

On 30 November 2022, OpenAI released ChatGPT to the public [1], a significant starting point in the widespread use of generative artificial intelligence (generative AI). The use of generative AI significantly influences computing education.

The ability of generative AI to generate responses in human-like language represents an advancement compared to earlier AI systems. This ability enables generative AI-based tools to effectively communicate with users, even for those with limited knowledge [2]. The language ability and being trained in computer science help AI-driven tools become great personal coding assistants for beginner programmers [3]. In two years, multiple AI programming assistants were developed and integrated into formal education to help students [4].

In education, generative AI is used in the form of large language models (LLMs)-based coding assistants. LLM-based coding assistants offer various features to help students in their studies. These features include real-time conversations, code completions, code debugging, and more features are being developed, making programming more accessible for beginners [5].

Initial studies have shown that a well-utilized programming assistant can be a great help for teachers and students. As generative AI tools have accelerated students' learning in problem solving and help avoid syntax errors [6]. On the other hand, a rapid integration of AI into educational contexts also raises concerns for educators and researchers. Zhai and Wibowo highlight a problem of students becoming over-reliant on AI-generated code, ignoring developing essential critical thinking skills [7]. Furthermore, there are concerns about the accuracy of the answers generated by the generative AI model, as AI models often provide incorrect answers or biases in the generated content [8].

Regardless of the effect, many students have utilized AI systems officially

or unofficially to assist them in their studies. In a survey of 2555 students, over half students reported using generative artificial intelligence in their study [9]. The report predicted that the number of students using these tools will only increase in the future. Therefore, there is a need to do more research into this topic.

The objective of this thesis is to analyze the evolving role and impact of generative AI-based programming assistants, or more specifically LLM-based tools, on beginner programmers and offer insight into an effective way to integrate them into an educational setting. More specifically, this thesis conducts a systematic literature review of the uses of generative AI-based programming assistants for novice programmers.

This thesis is divided into five sections. Section 2 introduces background information on generative AI and LLM-based coding assistants and prior literature about their application in programming education. Section 3 presents the purpose of the study and the model for the search strategy. Sections 4 and 5 describe the results and discussion, and Section 6 concludes the thesis.

2. Background

This section will first provide a technical explanation and background information about large language model-based coding assistants. Secondly, this section will discuss various applications of coding assistants in programming education.

2.1 Generative Artificial Intelligence and Large Language Model

Generative Artificial Intelligence refers to a sub-class of artificial intelligence systems that are designed to generate new data, such as texts, images, audio, video, and other forms of data. To do this, generative AI is built upon deep neural network architecture with multiple hidden layers to process and transform data to capture complex patterns from the large training data. To capture these complex patterns efficiently, generative AI runs in unsupervised or self-supervised learning approaches, allowing it to learn the pattern without labelling them. Generative AI then uses those patterns to generate new data with similar characteristics. [10]

In deep neural network architectures, the Transformer architecture is particularly relevant as it features an attention mechanism. Unlike other deep neural networks, the Transformer model utilizes self-attention mechanisms that allow the model to weigh the importance of different words in an input sequence to better understand the relation between them. This mechanism and the transformer architecture are the foundation for most modern large language models. [11]

Large language models (LLMs) are a subtype of generative AI that are trained specifically to generate natural language. LLM is typically developed through a two-phase process: pre-training and fine-tuning. Pre-trained LLMs are trained on large datasets of human-generated texts to capture general language knowledge, including sentence structure,

grammar, and syntax. While a pre-trained model can generate coherent human-like texts, the model can still be tailored to a specific task by fine-tuning. The fine-tuning process will improve the pre-trained model's ability in a specific field by training the model on domain-specific datasets. The additional training allows fine-tuned models to perform with greater accuracy and better performance than the general pre-trained model. The fine-tuned model was then used as a base for many generative AI applications, including programming assistants. [12]

Despite multiple training phases, LLMs can still exhibit "hallucination" behaviors. "LLM hallucination" refers to when an LLM's generated text sounds reasonable and confident but is factually incorrect or entirely made up. "LLM hallucination" is one of the current technical limitations of modern LLMs . [13]

2.2 Large Language Model-based Coding Assistants in Programming Education

One of the first noticeable LLM-based programming assistants is GitHub Copilot, which is powered by the LLM model OpenAI Codex. OpenAI Codex is an LLM that was specifically trained on natural language and then fine-tuned on a large database of code. The fine-tuning process gives Codex great code generation and code explanation capabilities [14]. Code generation and code explanation capabilities of Codex help GitHub Copilot perform a range of tasks, including code generation, code debugging, and code completion through conversations using natural language [15].

In addition to GitHub Copilot, there are other coding assistant products such as Amazon CodeWhisperer, developed by Amazon, and AlphaCode, developed by DeepMind [16], [17]. General-purpose generative AI, such as ChatGPT, can also be used as coding assistants. Multiple coding assistants available on the market make LLM-based programming assistants easily accessible for everyone.

Despite GitHub Copilot and other programming assistants being promoted as productivity technology for professional developers. Various studies are trying to explore the potential of LLM-based programming assistants in an educational context.

Elbanna et al. [18] perform a systematic literature review to investigate the uses of ChatGPT in personalized tutoring, automated grading and feedback, and interactive learning. The paper indicates that ChatGPT can

be used in boosting efficiency by automating routine tasks and generating learning material. The uses of ChatGPT can also improve students' learning experiences and familiarize them with the use of new technology.

Sarsa et al. [19] explore OpenAI Codex capabilities of generating programming exercises and additional code explanation for learning materials. The authors found that the generated content is high quality, and some is even ready for use. As a result, there are potential benefits to using Codex as a tool for instructors.

Leinonen et al. [20] explore using Codex to enhance programming error messages by generating additional explanations of the error messages, then try to generate suggested fixes for those errors. The results show the potential of using LLMs to enhance error messages by having up to 70% of the generated error messages' explanations be an improvement from the original error messages. However, the suggested fix correctness is only around 47%.

MacNeil et al. [21] using generative AI GPT-3 to generate additional explanations of given code snippets to improve students' learning experience.

Savelka et al. [22] evaluate the effectiveness of 3 different GPT models on how they answer post-secondary level programming multiple choice questions. The results indicate that there is a difference in successful rate between questions that have code snippets and those that do not contain code snippets. The best GPT model is text-davinci-003 with 77,9% correct for questions with code snippets and 59,5% correct for questions without code snippets.

The examples above present alternative applications of LLM-based coding assistants aside from serving as tools for students to use. Therefore, these papers offer other perspectives about coding assistants in programming education, supplementing this thesis's aim.

3. Systematic Literature Review Approach

3.1 Methodology and Research Questions

This paper uses "Preferred Reporting Items for Systematic reviews and Meta-Analyses" (PRISMA) in the selection process as it is the standard format in reporting systematic reviews. PRISMA enhances transparency in literature reviews by providing a 27-item checklist and a four-step flow diagram covering identification, screening, eligibility, and inclusion [23].

Figure 1 below is a four-step PRISMA diagram and is made using the tool provided from [24]. In the identification process, using the search query described above, we have identified 36 records from the ACM database, 24 records from the IEEE database, and 51 records from Scopus. Before screening, we excluded seven duplicate records, leaving 104 records for screening. In the screening process, we screened the titles and abstracts of articles, and it yielded 98 records. Regarding eligibility, we reviewed the methods, discussion, and results sections to determine if the records fit the inclusion and exclusion criteria. In the end, 22 records are included in the final reviews.

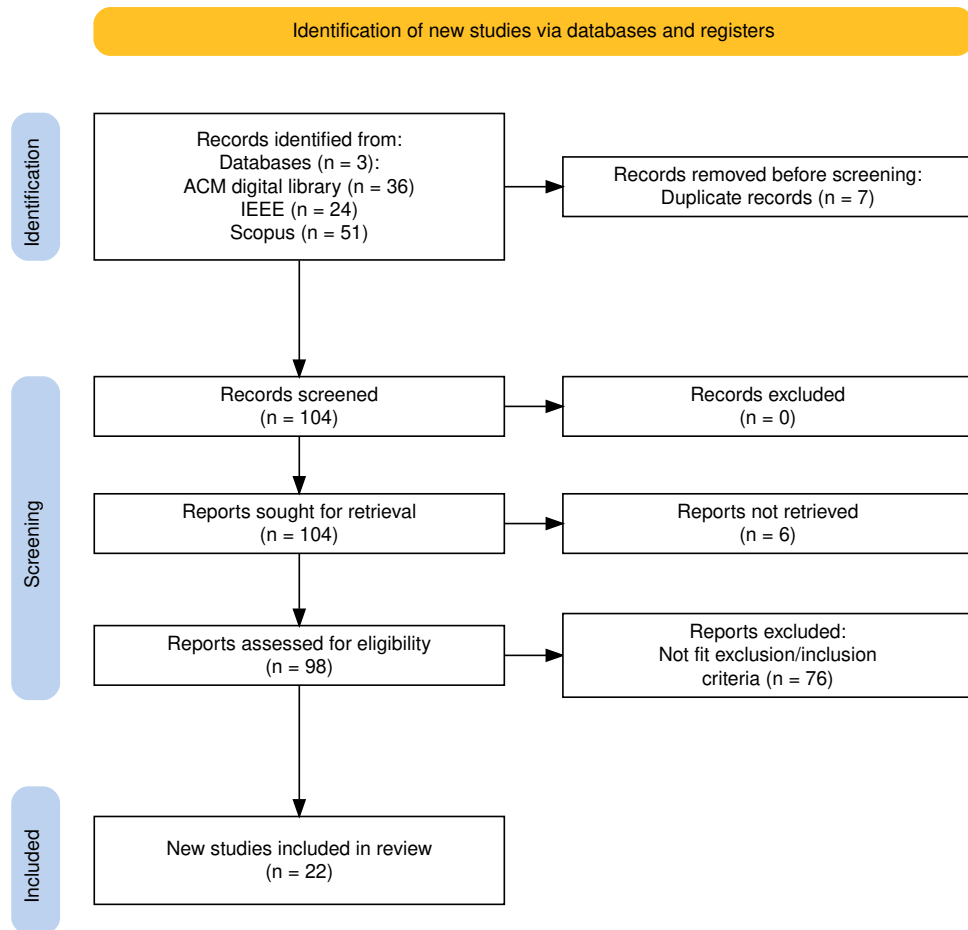


Figure 3.1. PRISMA diagram (A flowchart illustrating the identification, screening, eligibility, and inclusion of studies in the systematic review.)

This review answers these research questions:

RQ1 What is the current design of large language model-based programming assistants?

RQ2 How have LLM-based programming assistants been used in the research literature?

RQ3 What are the educational benefits for novice programmers using programming assistants?

RQ4 What are the educational downsides for novice programmers using programming assistants?

RQ5 What are the challenges in using coding assistants and the limitations of the tool?

3.2 Search Strategy

A systematic review was utilized to identify and select relevant articles. This research limited the search to studies that were published from 2022 to 2024 on three relevant databases: Scopus, IEEE, and ACM digital

library. The search query was created by using the main keywords from the research questions, such as "Programming assistant", "Large Language Model", and "novice programmer", listing all their synonyms, and shortened names. The search query is listed as follows:

("Programming assistant" OR "AI programming assistant" OR "Code generation model" OR "Program Synthesizers") AND ("Large Language Model" OR "Generative AI") AND ("programming education" OR "computing education" OR "computer science teaching") AND ("novice programmer" OR "beginner programmer" OR "student")

The search query was applied to the titles, abstracts, and full texts of the papers. The search was conducted in August 2024 across the three databases mentioned above.

3.3 Inclusion and Exclusion Criteria

To exclude other types of records that can appear in the search result, such as posters or advertisements, this thesis only includes papers that have five or more pages and are written in English. The thesis defined the generative AI-based programming assistant as a tool with the ability to generate natural language output and is designed to help developers in coding assignments by responding to queries made by such developers. With this definition in mind and a focus on beginner developers, this paper also filters papers with these specific conditions. First, the paper must have to present or discuss the usages of such AI tools. Second, the paper must relate to the teaching or learning of computer science or programming education. Finally, the paper must either present the tutoring system or describe interactions between students and programming assistants. In summary, the following inclusion criteria were followed:

- **Database:** Scopus, IEEE, ACM digital library
- **Publication period:** 2022-2024
- **Length of Paper:** 5 or more pages
- **Focus on LLM or generative AI:** Papers that present or discuss the use of generative AI or LLM.

- **Focus on computing education:** Papers that relate to teaching and learning of computer science and programming education.
- **Focus on student using programming assistant:** Papers that either present the tutoring system or describe the interaction between students and programming assistants

Exclusion criteria:

- **Not in English:** The paper is not written in English.
- **Not education:** The focus of the paper is not on education.
- **Not computing education:** The focus of the paper is not on programming education.
- **Not LLM coding assistant:** The focus of the paper does not include generative AI or LLM models.
- **Not for novice programmer or student:** The focus of the paper is on using generative AI or LLM models for students and novice programmers to use.
- **Not accessible:** Paper is not accessible.

3.4 Data Extraction Method

During the screening phase, to extract data from the selected papers, we used a data extraction form with ten questions, with each question corresponding to one of the research questions. The lists of 10 questions on the form could be seen in [Table 1.1](#) in [Appendix A](#) section.

The data extraction form was designed to extract relevant details from each paper reviewed systematically. The forms were created in Google Forms for easier data collection and visualization during and after the screening process. It included both quantitative and qualitative questions, each aligned with the research questions in this paper. Questions 1.1 to 1.5 provided metadata of the reviewed paper. Question 2 captures the specific

model of LLM. Question 3 captures different features implemented in the LLM-based coding assistant. Questions 2 and 3 combined were relevant to research question 1. Question 4 collects data about how the participants in the papers used the generative AI system by collecting the kind of queries they made to the system. This question correlates with research question 2. Questions 5 and 6 capture the upside and downside of the proposed coding assistant, corresponding to research questions 3 and 4. Finally, questions 7 and 8 capture the limitations of the system and challenges users feel when using it. Both correlate with research question 5. All the questions in data extraction form are listed as [Table 1.1](#) in [Appendix A](#)

4. Results

This section provides a descriptive overview of all the data extracted during the review. All 22 papers included in the literature review are listed in [Table 1.2](#) in [Appendix A](#). The section discusses the following subjects: metadata of reviewed papers, classification of the literature, classification of coding assistants, and quantitative findings of each research question.

4.1 Metadata of Reviewed Papers

This section analyzes the metadata of the reviewed papers.

[Table 4.1](#), [Table 4.2](#), and [Table 4.3](#) list metadata of all the experiments proposed in the reviewed papers.

As in [Table 4.1](#), all but one paper were published during 2023 and 2024, and of these 22, 15 papers were published in 2024.

Years the paper published	Number of papers
2022	1
2023	6
2024	15

Table 4.1. Published year of reviewed paper

[Table 4.2](#) also shows that most of the experiments in the reviewed paper originate from the US, with the second largest region being the EU, with 5 experiments were performed. Since some experiments and surveys were performed in multiple places, the number of locations mentioned is greater than the number of reviewed papers.

Location of proposed experiment	Number of papers
US	15
EU	5
Asia	4
South America	1

Table 4.2. Location of proposed experiment

The distribution of participants in experiments is displayed in [Table 4.3](#), most of the experiments will be conducted at a small scale (50 participants or less). Medium-scale experiment comprises 7 papers that were performed with 50 to 100 participants, and 3 other papers were performed on 100 to 200 participants. There are 3 experiments conducted on an extensive scale with more than 200 participants. The paper experiment with over 100 participants adopted the survey research method instead of scientific investigation.

Number of participant in proposed experiment	Number of papers
0-50	9
50-100	7
100-200	3
200 +	3

Table 4.3. Number of participants in proposed experiment

4.2 Classification of the Literature

The reviewed papers in this thesis fall into three broad categories: papers studying beginner programmers' usages of coding assistants and the impact of those usages, papers analyzing the capability and designs of coding assistants, and thirdly, survey and interview papers

Classification of Literatures	Number of papers
Papers studying beginner programmers' use of coding assistants and the impact of those usage	11
Survey, interview papers	6
Papers analyzing the capabilities and designs of coding assistants	5

Table 4.4. Classification of Literatures

4.2.1 Paper Studying Beginner Programmers' Uses of Coding Assistants and the Impact of Those Usage (12 papers)

Nearly half of the reviewed articles study the interactions between beginner programmers and coding assistants [25]–[35]. The majority of this group mainly focuses on the way beginner programmers use coding assistants [27]–[30], [33]–[35]. Papers [27], [29], [30], [35] pair beginner programmers with instructor-design coding assistants. Paper [28], [33], [34] conducts experiments between novices and general coding assistants.

The other group of papers mainly analyzes the impact of coding assistants on users' study performance [25], [26], [31], [32]. The detailed impact of using coding assistants will be discussed later in [section 4.6](#) and [section 4.7](#).

4.2.2 Survey and Interview Paper (6 papers)

Six of 22 papers [36]–[41] are papers that conduct surveys or interviews about coding assistants and novice programmers. Paper [36]–[38], [41] survey students' uses of coding assistants or generative AI. All discuss both the usage of the tools and the perceptions and trust of students about AI coding assistants. Some papers express slightly positive perceptions of the uses of AI tools in education [36], [37]. However, the majority of students additionally express concerns about academic integrity and risks to learning [36]–[38]. Paper [41] explores the relation between students' trust, perceived usefulness, and ease of use of coding assistants.

Paper [39] and [40] both interview instructors about the uses of generative AI in school. Paper [39] focuses on teachers' perceptions of ChatGPT and evaluates ChatGPT's capabilities in various applications, including coding assistants. Paper [40] focuses on instructors' plans to adapt to ChatGPT in both the short-term and long-term. The results of [40] indicate

that there are two approaches to adapt to the AI code generation tool in the long term. The first approach is to resist, and the other approach is to accept and integrate AI code generation tools into education.

4.2.3 Paper Analyzing the Capability and Designs of Coding Assistants (5 papers)

Five of 22 papers [42]–[46] are experiments that analyze the capability and design of coding assistants, where students use the tool and provide feedback. Four of five papers were testing instructor-design coding assistants with different features, such as prompt-less features in [42], [46], context integration in [43], [45], [46], guardrails, and prompt validation in [45], [46]. Paper [44] performs an experiment where students test five code generation tools, including Copilot, to determine how the tool design can affect learnability and usability.

4.3 Classification of Large Language Model-based Coding Assistant

In the reviewed papers, this thesis has identified two main categories of LLM-based coding assistants: General coding assistants and instructor-design coding assistants.

General Coding Assistants

General coding assistants are publicly available coding assistants that are either general-purpose purpose like ChatGPT, or coding-specific but not educationally focused, like GitHub Copilot. Because of the fast pace in the development of generative AI, information about specific model versions quickly becomes outdated at the time of this paper. Therefore, this thesis considers all these tools under the category of their main name, such as ChatGPT models or GitHub Copilot.

Instructor-design Coding Assistants

From the reviewed paper, there are a variety of coding assistants that have been developed by teachers to fit the educational environment. These instructors design coding assistants that utilize the natural language capabilities of generative AI, combined with a customized front-end tailored for educational uses. Unlike general coding assistants, instructor-design coding assistants can have additional features. These features can be prompt-less suggestions, additional instruction rules to generate the an-

swer, or can be trained locally through the course material for higher quality. In total, seven instructor-design coding assistants have been identified among the reviewed papers. These include: GILT [42], AI Knowledge Assistant [43], CODEHELP [27], [45], NetLogo Chat [29], ASKGPT [30], AI-Tutor [34], Charlie the Coding Cow [35] , CodeAid [46].

4.4 RQ1 What Is the Current Design of Large Language Model-based Programming Assistants?

To understand the current design of LLM-based coding assistants, this thesis looks at all the current features that are available in both general coding assistants and instructor-design coding assistants

Mentioned feature of generative AI system	Number of papers mentioned
Information presenting features: Interactive chat	12
Information presenting features: One-time chat	3
Information presenting features: Prompt-less suggestion	2
Quality features: Guardrails	4
Quality features: Code context incorporation	4
Quality features: Prompt validation	3

Table 4.5. Mentioned feature of generative AI system

Table 4.5 summarizes the main features of generative AI systems across the reviewed studies. There are two main categories of features tested in the reviewed papers: information presenting features and quality features. Information-presenting features are present in every coding assistant as a way for students to get information or answers from the model. Quality features found specifically in instructor design coding assistants as an improvement from the general programming assistant to produce higher quality responses or to fit educational uses.

Information-presenting features

Regarding information-presenting features, this thesis has identified several themes.

- **Interactive chat** Interactive chat refers to a feature in which users

can interactively ask questions or converse with the model. Interactive chat allows students to interact with the assistants through natural language input and receive responses with low delay. Using interactive chat, students can get support with various tasks such as fixing code, gathering information, and explaining material. Interactive chat is mentioned in [25], [26], [28], [29], [31], [32], [35], [36], [39], [42], [43], [46]

- **One-time chat** One-time chat only allows users to ask questions one time, then the model will generate answer but no further follow up is available. The mentioned reason behinds one-time chat is to guide student toward communicating effectively and providing all relevant information in one form instead of a single text input. One-time chat feature present in [27], [30], [45]
- **Prompt-less suggestions** Prompt-less suggestion features can provide helpful information to users without having them type in their prompt. Prompt-less suggestions can come in two variations: automatic code suggestions like Github Copilot's code completion feature, or a button that encodes frequently asked questions. This features appear in [30], [33], [34], [42]

These information-presenting features highlight a growing trend in researching the best design interaction between coding assistants and novice programmers. One research trend would be between one-time chat and interactive chat. Interactive chat offers more freedom and ease of use in interacting with coding assistants, but is not as effective in encouraging meta-cognitive engagement as one-time chat. One-time chat requires users to think carefully for each prompt, but guides them more effectively by having them fill in a form. [33], [46]

Another emerging design would be prompt-less suggestions. Prompt-less suggestion design helps circumvent the problem of requiring users to know prompt engineering techniques. Early studies have shown that students prefer prompt-less suggestions more than interactive chat due to its effectiveness, learnability, and convenience [34], [42], [44]

Quality features

Quality features are features that are additionally added in instructor-design coding assistants to enhance the quality of generated responses or

enhance users' experience. For quality features, we came up with several categories.

- **Guardrails** Guardrails features are features that specifically place constraints on how the model can generate responses to student questions. The most common constraints are placed to prevent the model from directly generating code snippets. Instead of generating solution code directly, the model tries to guide the student using natural language or pseudo-code instead. Guardrails feature present in [27], [34], [45], [46]
- **Code context incorporation** Code context incorporation features help to improve coding assistants' responses by integrating code context straight into prompts. These code contexts often include information that students usually do not give to the mode, such as students' understanding, assignment details. and course details. Code context incorporation feature is included in [27], [42], [43], [45]
- **Prompt validation** Prompt validations evaluate whether the user's initial input contains sufficient information for the coding assistants to generate a meaningful and helpful answer. If the input is found to have insufficient information for the model to work with, instead of assuming information, like in the case of general coding assistants, the system will ask the user to provide additional details or clarification before proceeding. Prompt validation is implemented in [27], [29], [45]

Each of the quality features is designed to circumvent a specific problem discussed in [section 4.7](#) and [section 4.8](#).

Guardrail features are designed to prevent the issue of overreliance. As will be discussed in [section 4.5](#), beginners often use coding assistants as a quick way to generate answers without attempting to solve the problem themselves. Guardrails can prevent this situation by restricting LLMs from directly generating the solutions, but instead try to guide students to solve the problems by offering them support through providing pseudo code or natural language instructions . [26], [27], [29], [45]

Code context incorporation and prompt validation attempt to prevent the limitation of beginners and coding assistant misreading each other, where this limitation will be discussed in detail in [section 4.8](#). Often, beginners do not provide enough information for coding assistants to generate responses

that fit their needs. [42], [43]

With code context incorporation, the system will have access to more information that beginners usually forget to send to the model. Therefore, the model can generate higher-quality responses with more information.

On the other hand, prompt validation will ensure that coding assistants will have enough information to work with. As a result, users will get better responses from the model and learn to use the coding assistant more effectively. [27], [45]

4.5 RQ2 How Have Large Language Model-based Programming Assistants Been Used In The Research Literature?

This thesis looks at queries to coding assistants to understand more about what tasks novice programmers use coding assistants for. A query to coding assistants is defined as the input that the user gives to the model.

Queries make by novice programmers	Number of papers mentioned
Code generation	12
Code debugging	8
Gathering information	8
Code explanation	7
Summarizing material	2
Generating feedback	2

Table 4.6. Type of queries novice programmers make

Table 4.6 summarizes different types of queries novice programmers make to coding assistants. There are six main tasks that students use coding assistants for: Code generation, code debugging, gathering information, code explanation, summarizing material, and generating feedback.

- **Code generation** Novice programmers use coding assistants to generate code snippets based on the specification provided by users. The uses of code generation are mentioned in [25]-[27], [29]-[31], [33], [36]-[38], [42], [46].
- **Code debugging** Novice programmers use coding assistants to identify and suggest corrections for their code. The uses of code debugging are mentioned in [25], [27], [29], [31], [32], [38], [45], [46].

- **Gathering information** Novice programmers use coding assistants to search and explain programming knowledge and concepts such as syntax, algorithms, and data structures. The uses of gathering information are mentioned in [27], [30], [31], [37], [38], [42], [45], [46].
- **Code explanation** Novice programmers use coding assistants to provide overall or line-by-line explanations for input code snippets. The uses of code explanation are mentioned in [25], [32], [37], [38], [42], [45], [46].
- **Summarizing material** Novice programmers use coding assistants to provide a condensed summary of learning material or online information. The uses of summarizing material are mentioned in [36], [38].
- **Generating feedback** Novice programmers use coding assistants to generate feedback on input code or solutions approach. The uses of generating feedback are mentioned in [34], [36].

From data of Table 4.6, novice programmers mentioned using coding assistants for code generation tasks with high frequency. Gathering information and code debugging come as the second most frequent requests mentioned. It shows that beginner focus more on receiving immediate help with programming problems rather than improving their coding skills and knowledge base. This trend is also noticed among instructors and is being mentioned directly in [27], [45].

4.6 RQ3 What Are the Educational Benefits for Novice Programmers Using Programming Assistants?

Proposed benefits for novice programmers	Number of papers mentioned
Efficiency and productivity	14
Skill and knowledge improvement	7
Enhanced learning experience	6
Psychological benefits	3
Accessibility	2
Prepare for future career	2

Table 4.7. Proposed benefits for novice programmers

From [Table 4.7](#), there are a variety of educational benefits for novice programmers to use LLM-based coding assistants. Most of the benefits are mentioned by students in surveys. We have identified six main themes of proposed benefits.

- **Efficiency and productivity** Coding assistants help programmer improve their efficiency by spending less time on multiple tasks such as looking for information, finding approaches to the problem, identifying, and suggesting solutions to error messages. Efficiency and productivity benefits are mentioned in [\[26\]](#), [\[29\]](#)-[\[31\]](#), [\[33\]](#), [\[36\]](#), [\[37\]](#), [\[39\]](#), [\[40\]](#), [\[42\]](#), [\[44\]](#)-[\[47\]](#).
- **Skill and knowledge improvement** Coding assistants can help programmer improve their skills and knowledge by providing detailed explanations, offering examples of code snippets, and programming concepts. Skill and knowledge improvement benefits are mentioned in [\[25\]](#), [\[26\]](#), [\[30\]](#), [\[33\]](#), [\[36\]](#), [\[44\]](#), [\[46\]](#).
- **Enhanced learning experience** Coding assistants can improve students' learning experience by being always available to guide students when they get stuck, or generating personalized feedback and review. Enhanced learning experience benefit is mentioned in [\[26\]](#), [\[36\]](#), [\[39\]](#), [\[45\]](#)-[\[47\]](#).
- **Psychological benefits** Coding assistants are reported to help reduce stress, anxiety, and frustration. Psychological benefits are mentioned in [\[26\]](#), [\[30\]](#), [\[33\]](#).
- **Accessibility** Coding assistants' 24/7 availability is a great help for students who may not have access to office hours or are too shy to ask for help in person. Accessibility benefit is mentioned in [\[45\]](#), [\[46\]](#).
- **Prepare for future career** Coding assistants help familiarize novice programmers with modern technologies used in the industry, and prepare them for future jobs. This benefit is mentioned in [\[40\]](#), [\[47\]](#).

From data listed in [Table 4.7](#), the most common proposed benefit is increases in efficiency and productivity. This trend aligns with the usage trend

discussed in **RQ2** mentioned in [section 4.5](#), where novice programmers mostly used coding assistants for code generation and code debugging tasks. Code generation and code debugging tasks directly result in shorter time spent solving programming assignment, which correlate to better efficiency and productivity.

Five papers have tried to determine whether the uses of coding assistants affect students' grades and class performance. The results of the five papers also varied from each other. [27], [31], [32] determined that using coding assistants does not affect the overall performance of students. While [25]'s experiment results indicate that higher frequency uses of LLM-based coding assistants for tasks such as code generation and code debugging lead to users having a lower grade. However, [25] results also suggest that the use of coding assistants for gathering information tasks does not affect students' overall grade. Finally [26] suggests that having access to coding assistants makes students perform slightly better on tests.

From all the conclusions above, it suggests that using a coding assistant for code generation and code debugging tasks appears to reduce the time spent solving assignments. However, there is no clear evidence that using coding assistants leads to improvement in skills or deeper understanding. Therefore, students could mistakenly perceive that their skills have been improved, when in fact, the performance gains are largely attributed to the help of coding assistants.

4.7 RQ4 What Are the Educational Downsides for Novice Programmers Using Programming Assistants?

[Table 4.8](#) summarizes different downsides for novice programmers using LLM-based coding assistants.

Proposed downside for novice programmers	Number of papers mentioned
Overreliance	14
Mislead by wrong responses	8
Academic misconduct	7
Impeded skill development	6

Table 4.8. Proposed downside for novice programmers

This thesis has identified four main downsides for students when they use coding assistants.

- **Overreliance** Novice programmers rely heavily on coding assistants, using generated code as a solution to assignments without fully understanding the underlying logic. This overreliance leads to novice programmers bypassing practicing critical thinking and problem-solving skills. As a result, novice programmers may struggle to develop the necessary skills and knowledge to solve further assignments and have to rely even more on coding assistants. Overreliance problem is discussed in [25]-[27], [29]-[31], [33], [34], [38]-[40], [45], [47].
- **Mislead by wrong responses** LLM-based coding assistants do not always produce correct responses or hallucinate, where they make up information. Novice programmers, who lack the knowledge and skill to verify the output, may be misled into learning incorrect information. in [29], [33], [35], [39], [44]-[47].
- **Academic misconduct** Novice programmers using coding assistants for unauthorized purposes, such as cheating in exams, plagiarism due to submitting AI-generated code without the student's contribution. This problem is discussed in [30], [31], [33], [38]-[40], [47].
- **Impeded skill development** Excessive use of LLM-based coding assistants can hinder the skill development of novice programmers. The reasons for this hindrance are that novice programmers often use coding assistants for key tasks such as code debugging or code writing to coding assistant, reducing opportunities to develop these skills. Impeded skill development downside is discussed in [25], [29], [31], [33], [39], [47].

The most cited downside for students is overreliance. This reflects the trend of students relying too heavily on coding assistants to solve the problem for them. This trend is the same as the one reflected in the previous two research questions in [section 4.5](#) and [section 4.6](#).

All reported downsides highlight a need for guidance for novice programmers and management of students' use. There are a variety of measurements that have been implemented to try and circumvent these downsides:

- **Detailed guideline** for using coding assistants can help reduce misuse and promote academic integrity [25], [38].

- **Guardrails** to prevent generate direct solution help reduce overreliance problem [27], [34], [45], [46].
- **Paired LLM tools with instructor feedback** can help reduce blind trust in coding assistants, reducing instances where students are misled by incorrect responses [27], [38], [46].
- **Redesigning assignment** to fit the uses of coding assistants, such as assignment focusing more on code understanding, can help reduce the overreliance problem while promoting skills development [32], [40].
- **Scaffolding AI responses** can help students understand AI responses better, supporting deeper programming understanding and less blindly trusting [48].

These measurements reflect two significant trends in how to integrate coding assistants into educational settings effectively. The first trend is the need for more detailed guidelines and new learning material. These will help the student to use the coding assistant in a way that supports learning. The second trend is the future development trend for LLM-based coding assistants. Researchers are moving away from general coding assistants and into instructor-design coding assistants. Unlike general coding assistants, instructor-designed coding assistants can take advantage of customized features, such as guardrails, to limit potential downside and challenges in using the tools.

4.8 RQ5 What Are the Challenges in Using Programming Assistants and Limitations of the Tool?

Technical Limitation of LLM-based Coding Assistants

Technical limitation	Number of papers mentioned
Inaccuracy and hallucination	15
Privacy and ethical concern	5
Usability issues	3
Limited context awareness	3

Table 4.9. Technical limitation of LLM-based coding assistants

Table 4.9 summarizes different technical limitations of LLM-based coding assistants mentioned by both novice programmers and instructors. This thesis has identified four major themes for the technical limitations of coding assistants based on the literature review.

- **Inaccuracy and hallucination** LLM-based coding assistants can generate incorrect or misleading responses, or even make up incorrect information (AI hallucination). These inaccuracies can confuse beginners and lower beginners' trust in coding assistants. This limitation is mentioned in [28], [29], [31], [33], [35]–[37], [39], [40], [42]–[47]
- **Privacy and ethical concern** LLM-based coding assistants may be trained on copyrighted data and generate responses without citation, raising concerns about plagiarism. In addition, there are security and privacy risks that coding assistants pose to students' data. This limitation is mentioned in [30], [33], [39], [40], [47]
- **Usability issue** LLM-based coding assistants' user interface is reported to be difficult to use or not beginner-friendly. This limitation is mentioned in [30], [36], [43]
- **Limited context awareness** LLM-based coding assistants perform poorly on context-sensitive prompts. The system's responses are weakest when it comes to course-specific code snippets or code understanding from multiple files. This limitation is mentioned in [30], [34], [44]

The most mentioned limitation of coding assistants is accuracy and hallucination, highlighting a core problem of coding assistants: LLMs can generate inaccurate information with high confidence. This core problem also aligns with the downside of being misled by wrong responses discussed in **section 4.7**.

Usability issues also promote the need for research for a better way to interact with coding assistants that are beginner-friendly and suitable in educational settings [29]. At the moment, most coding assistants attempt to use a basic design of an interactive chat with a box to input a prompt. However, instructor-design coding assistant and GitHub copilot have experimented with different designs, such as prompt-less features or integrated into IDE as discussed in **section 4.4**.

Challenges Faced by Students Using Programming Assistants

Challenges faced by student using coding assistants	Number of papers mentioned
Beginners and coding assistants misreading each other's	8
Unstructured/Fragmented learning	6
Hard to detect error in coding assistants' responses	4

Table 4.10. Challenges faced by students using coding assistants

Table 4.10 shows three main key challenges when interacting with LLM-based coding assistants. These challenges were identified from both survey-based and experimental studies. Four main challenges are:

- **Beginners and coding assistants misreading each other's** Novice programmers lack a clear strategy and prompt engineering skills to craft an effective prompt to get coding assistants to generate answers that align with their needs. As a result, novices often feel that coding assistants are misunderstanding their requests and generating responses that do not match student expectations. This challenge is mentioned in [26], [27], [29], [35], [39], [42], [44], [45].
- **Unstructured/Fragmented learning** LLM-based coding assistants often generated responses without considering the user's current level of knowledge. As a result, users may receive answers that are outside of their understanding, with no contextual link to concepts that they have learned. The outcome of this is students having fragmented learning. This challenge is mentioned in [29], [30], [33], [39], [40], [45].
- **Hard to detect error in coding assistants' responses** Many students, especially novice programmers, lack the skills and knowledge to verify the correctness of the generated answers from coding assistants. As a result, they often accept the generated answer blindly or are misled by wrong information. This challenge is mentioned in [29], [36], [40], [45].

The challenges of unstructured/fragmented learning can be solved by providing more information about the users and their level of study. This ap-

proach has been done by implementing code context incorporation features, discussed in [section 4.4](#).

The most mentioned challenge is the issue of beginner and coding assistants misreading each other. Based on the reviewed paper, there are multiple reasons behind these challenges.

Paper [26]-[29] report that students tend to spend low effort on crafting their prompts. Paper [26] results show that 32% of prompts submitted are the same as assignment details. [27] reports that most students do not modify their prompt that much, writing too little or nothing, and often omitting important details when requesting help. Paper [28], [29] analyze the cause more deeply, pointing out that the cause of poor crafting prompts can be due to a lack of technical knowledge and experience in programming. As a result of these low-effort prompts, the assistant does not have enough information to match the user's requests, therefore creating low-quality or mismatched answers.

On top of that, due to the lack of information from the user's prompt, LLM-based coding assistants have to assume missing information based on popular patterns. This causes the assistant to generate code that looks correct but is not fit for the user's requests. This results in beginners feeling like coding assistants misunderstand their intentions . [27], [29], [44]

Overall, the challenge of beginners and coding assistants misreading each other reflects a communication gap between novice programmers and LLM-based coding assistants. To solve this problem, [27], [29], [44] had tried to implement prompt validation features, which were discussed in [section 4.4](#). Another solution would be to give more guidance to students on how to use coding assistants effectively by teaching them prompt engineering skills.

5. Discussion

This section provides broader implications about patterns identified in the results, key research trends, and differences found in the reviewed papers. This section will discuss these topics in order: summarizing answers to research questions, implications for learning programming, implications for programming educators, and recommendations for integrating coding assistants.

5.1 Summarizing Answers to Research Questions

In this section, this thesis will summarize the key points and results of each research question in order.

- **RQ1 What Is the Current Design of Large Language Model-based Programming Assistants?**

The current design of coding assistants includes research in two main feature categories: information-presenting features and response quality features.

All general coding assistants use interactive chat as their main information-presenting feature. However, more and more research is exploring different means for novice programmers to interact with LLM-based coding assistants. These different variations of interactive chat comprise One-time chat and prompt-less suggestions. Each of these variation try to solve different challenges in using coding assistants, though this often comes at the expense of the tool's ease of use [33], [46].

Alongside experimenting with information presenting features, researchers are also exploring different quality-improving features such as guardrails, prompt validation, and code context incorporation to circumvent the challenge of using coding assistants.

- **RQ2 How Have Large Language Model-based Programming Assistants Been Used In The Research Literature?**

Novice programmers mainly used LLM-based coding assistants for code generation and code debugging, along with other uses such as information gathering, code explanation, summarizing material, and generating feedback.

The usage trend shows that students prefer using coding assistants for immediate help over improving their knowledge and skills. This trend suggests that students treat coding assistants more like a machine that generates answers rather than a tool to help them study better.

- **RQ3 What Are the Educational Benefits for Novice Programmers Using Programming Assistants?**

The majority of reported benefits are efficiency and productivity, along with skills and knowledge improvement. However, several studies show that using coding assistants does not affect students' academic performance, but only shortens their time spent solving assignments [27], [31], [32]. Because of that, the benefits reported seem more aligned with the case of students mistakenly attributing the capabilities of a coding assistant to their improvement.

- **RQ4 What Are the Educational Downsides for Novice Programmers Using Programming Assistants?**

Four downsides have been identified, these are: overreliance, being misled by wrong responses, academic misconduct, and impeded skill development.

Overreliance is the most cited downside. This fact aligns with the trend of students relying on coding assistants to solve the problem for them like as discussed in RQ2 and RQ3

Several measurements have been applied to both the coding assistants and novice programmers to minimize these downsides. On the coding assistants' side, there is a need for quality-improving features such as guardrails and scaffolding AI responses to mitigate the overreliance problem, but still allow the model to provide useful support to users.

On the novice programmer side, there is a need for more detailed guidelines and a new kind of assignment that takes into account the uses of coding assistants.

- **RQ5 What Are the Challenges in Using Programming Assistants and Limitations of the Tool?**

LLM-based coding assistants have multiple limitations that can be improved to be higher quality and better fit educational usages. These limitations include: inaccuracy and hallucination, privacy and ethical concerns, usability issues, and limited context awareness

Beginners also face multiple challenges when using coding assistants. These challenges include beginners and coding assistants misreading each other, fragmented learning, and hard-to-detect errors in coding assistants' responses.

Inaccuracy and hallucination issues of coding assistants are especially dangerous for novice programmers, who do not have enough skills and knowledge to verify the correctness of LLMs' output. As a result, beginners can easily be misled by wrong responses.

Beginner and coding assistants misreading each other is a big challenge in using coding assistants. This challenge is a combination of multiple compounding challenges of lower-quality prompts by users, resulting in general coding assistants assuming missing information.

Overall, there is a need to address the communication gap between novice programmers and LLM-based coding assistants so that novice programmers can best use coding assistants.

5.2 Implications for Learning Programming

In this section, this thesis will propose implications for learning programming regarding the preferred coding assistant type, recommended teaching model, managing students' trust in coding assistants, and prompt engineering skill.

- **Instructor-design Coding Assistants Outperform General Coding Assistants**

Findings from **RQ4** and **RQ5** show that coding assistants have many downsides and limitations. However, unlike general coding assistants, instructor-design coding assistants were actively built with features to circumvent these downsides and limitations. One successful example is the guardrails function. Guardrails aim to reduce the issue of over-reliance while still offering support to students [26], [27], [29]. These

customized features make instructor-design coding assistants more suited for classroom use than general coding assistants [49].

The best way to avoid the downside of using LLM-based coding assistants is to guide students to use instructor-design coding assistants over the easy-to-get solutions generated from general coding assistants. In order to do that, students need to be aware of the potential downsides of using LLM-based coding assistants and why instructor-design coding assistants are the best for them.

- **AI-human Hybrid Teaching Model**

Based on the answer found to **RQ3** and **RQ5**, a good practice to integrate coding assistants into education would be to use them primarily for simple questions. With this practice, students can still make use of LLM-based coding assistants' benefits without being exposed to the problem of hallucination and incorrect information [46]. Additionally, simpler outputs make it easier for students to verify themselves.

However, allowing beginners to freely use AI without supervision poses a risk of them being misled by inaccurate responses. Therefore, teachers' feedback and supervision will continue to be instrumental. As a result, a model where coding assistants offer help to easily answer questions, pairing with human feedback on tasks solved, could be a good way to integrate coding assistants. This model has been tested in three papers [27], [45], [46] and has positive results.

- **Managing Trust in LLM-based Coding Assistants**

Results from **RQ4** and **RQ5** highlight negative effects when students blindly trust AI's responses, especially in cases where they lack the skills and knowledge to verify the responses. Novice programmers need to be taught that AI can be wrong, even though the generated responses come with a high confidence tone. Therefore, information about the upsides and downsides when working with programming assistants should also be incorporated into teaching material [40].

Not only blindly trusting but also solely relying on coding assistants for information gathering is further not ideal. Best practices would be double-checking answers from different sources, like course materials or online websites. This way, there would be less chance of becoming

overrelying on AI. [31]

- **Prompt Engineering Is a Critical Skill but Being Underdeveloped**

The issue of low-quality prompts and the problem of beginner and coding assistants misreading each other highlight the need for developing prompt engineering skills in novice programmers [30]. However, prompt engineering skill is considered underdeveloped, and most of the research papers show that students did not receive guidance on how to strategy the best way to communicate with coding assistants. Therefore, teachers should consider adding prompt engineering strategies into official course materials and creating prompt design exercises to help beginner programmers develop this skill. [50], [51]

5.3 Implications for Programming Educators

In this section, this thesis will provide insight into implications for programming educators regarding the need for guidelines, redesigning assignments, and future development direction for coding assistants.

- **Clear Guidelines for Using Coding Assistants Effectively and Ethically**

Based on the results of **RQ4**, there is a need for detailed guidelines on how students should use coding assistants. Without clear rules and guidance, students will misuse coding assistants, leading to academic misconduct, such as using them to complete entire assignments. Conversely, without clear guidance, students are too afraid of using coding assistants due to uncertainty and risks of ethical violations [30], [38].

Instructors and teaching institutions should provide strict and transparent guidelines about what is acceptable and what is not. Doing this will help maintain academic integrity. [52]

- **Redesign Assignment with Consideration of AI**

The usage trend identified in **RQ2** indicates that different kinds of assessments are needed that account for an AI coding assistant. The

current assignment type of writing code that passes automatic tests is easily solved by coding assistants [40], [50], [53]. This type of assignment often encourages overreliance on AI and is a problem since it is easy to ask coding assistants to just generate the answers.

Educators need to design new types of assignments that are AI-proof, such as code understanding assignments, image-based assignments, and paper-and-pencil exams [32], [40]. By doing so, it should be possible to limit the misuse of coding assistants.

- **Develop Further Instructor Design Coding Assistants**

The literature review and the answer to **RQ1** show that the instructor-design coding assistants have a clear advantage over general coding assistants like ChatGPT or GitHub Copilot. While general coding assistants are powerful, they are less suitable for educational uses than instructor-design coding assistants. Instructor-design coding assistants can have multiple customization features such as Guardrails, prompt validation, and code context incorporation. These customizations help avoid some of the main downsides and challenges in using coding assistants.

To successfully integrate LLM-based coding assistants into the classroom, more high-quality customizations for instructor-design coding assistants are needed. Alongside that, the long-term effects of these customizations should be researched. Further research should establish which customizations best support the students

5.4 Recommendation for Integrating Large Language Model-based Coding Assistant

In this section, this thesis will provide different recommendation practices derived from the above implications to integrate coding assistants effectively into an educational setting. These recommendations comprise:

- **Encourage students to use instructor-design coding assistants** with guardrails and other features for safer and more effective uses.
- **Use hybrid teaching model** where coding assistant support easy-to-answer tasks while under instructor's supervisors and feedback

- **Integrate prompt engineering skills** into teaching material to help students communicate effectively with LLM-based coding assistant
- **Establish clear AI usage policies and guidelines** inside educational institutions to avoid academic misconduct.
- **Design AI-proof Assignment** to replace the current assignment that is easily solved by coding assistants.

5.5 Limitations of the Study

This thesis has three main limitations. The first concern was the time frame of the study selected for the literature review. This thesis performed a literature review at the end of the summer of 2024. Therefore, research beyond this time mark will not be reviewed. As a result, up-to-date research trends and data may be different from those presented in this thesis.

Secondly, due to rapid change in the field, many research papers on this topic are published in non-traditional channels such as arXiv. These were excluded from the literature review process.

Third, the review included no formal cross-check with existing literature reviews in the same domain. Furthermore, this thesis performed no quality assessments for the reviewed papers using quality metrics. As a result, this lack of quality assessment may impact the reliability of some reviewed papers.

6. Conclusion

This thesis focused on understanding the impact of LLM-based coding assistants on novice programmers. This goal is achieved by investigating five key aspects corresponding to five research questions:

RQ1 What is the Current Design of large language model-based Programming Assistants?

RQ2 How have LLM-based programming assistants been used in the research literature?

RQ3 What are the educational benefits for novice programmers using programming assistants?

RQ4 What are the educational downsides for novice programmers using programming assistants?

RQ5 What are the challenges in using coding assistants and the limitations of the tool?

To understand more about these five key aspects, a systematic literature review was conducted using publications that include interaction between LLM-based coding assistants and novice programmers or students.

The key results of the review highlight several important trends. First, novice programmers use coding assistants mainly for immediate help, rather than for improving their skills and knowledge. This usage trend leads to several challenges and downsides, with the most prominent one being the overreliance problem. To circumvent these problems and challenges, instructor-design coding assistants offer multiple custom features and mechanics, such as the guardrails feature. As a result, instructor-design coding assistants show more positive outcomes compared to general coding assistants like ChatGPT or GitHub Copilot. On the other hand, LLM-based coding assistants show that they have their unique benefits, such as their availability and improving the learning experience.

In conclusion, while LLM-based coding assistants offer real opportuni-

ties to support novice programmers in their learning, their effectiveness depends greatly on how beginners use the tools. For the successful integration of LLM-based coding assistants into a classroom, there needs to be more proper guidance to ensure that novice programmers understand the downsides and the correct way to use the tools. Alongside that, there needs to be more customizations for instructor-designed coding assistants to limit the downside of using the tools while still taking advantage of their benefits.

Bibliography

- [1] OpenAI, *Introducing chatgpt*, Online, Accessed: 2025-03-02. [Online]. Available: <https://openai.com/index/chatgpt>.
- [2] OpenAI, *Introducing chatgpt*, <https://openai.com/index/chatgpt/>, Accessed: Jun. 16, 2024.
- [3] M. Q. Feldman and C. J. Anderson, "Non-expert programmers in the generative ai future," in *Proceedings of the 3rd Annual Meeting of the Symposium on Human-Computer Interaction for Work*, 2024, pp. 1-19.
- [4] Z. Bahroun, C. Anane, V. Ahmed, and A. Zacca, "Transforming education: A comprehensive review of generative artificial intelligence in educational settings through bibliometric and content analysis," *Sustainability*, vol. 15, no. 17, p. 12 983, 2023.
- [5] S. S. Sengar, A. B. Hasan, S. Kumar, and F. Carroll, "Generative artificial intelligence: A systematic review and applications," *Multimedia Tools and Applications*, pp. 1-40, 2024.
- [6] J. Prather *et al.*, "The widening gap: The benefits and harms of generative ai for novice programmers," 2024.
- [7] C. Zhai, S. Wibowo, and L. D. Li, "The effects of over-reliance on ai dialogue systems on students' cognitive abilities: A systematic review," *Smart Learning Environments*, vol. 11, no. 1, p. 28, 2024.
- [8] B. A. Becker *et al.*, "Programming is hard - or at least it used to be," in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, Mar. 2023. DOI: [10.1145/3545945.3569759](https://doi.org/10.1145/3545945.3569759).
- [9] H. Johnston, R. F. Wells, E. M. Shanks, T. Boey, and B. N. Parsons, "Student perspectives on the use of generative artificial intelligence technologies in higher education," *International Journal for Educational Integrity*, vol. 20, no. 1, p. 2, 2024.

- [10] S. Feuerriegel, J. Hartmann, C. Janiesch, and P. Zschech, "Generative ai," *Business & Information Systems Engineering*, vol. 66, no. 1, pp. 111–126, 2024.
- [11] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.
- [12] K. Sun and M. Dredze, "Amuro & char: Analyzing the relationship between pre-training and fine-tuning of large language models," *arXiv preprint arXiv:2408.06663*, 2024.
- [13] L. Huang, W. Yu, W. Ma, *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.
- [14] OpenAI, *Openai codex*, Accessed: 2025-03-28. [Online]. Available: <https://openai.com/index/openai-codex/>.
- [15] GitHub, *What is github copilot?* Accessed: 2025-03-28. [Online]. Available: <https://docs.github.com/en/copilot/about-github-copilot/what-is-github-copilot>.
- [16] A. W. Services, *What is codewhisperer?* <https://docs.aws.amazon.com/codewhisperer/latest/userguide/what-is-cwspr.html>, Accessed: 2025-03-28.
- [17] DeepMind, *Competitive programming with alphacode*, <https://deepmind.google/discover/blog/competitive-programming-with-alphacode/>, Accessed: 2025-03-28.
- [18] S. Elbanna and L. Armstrong, "Exploring the integration of chatgpt in education: Adapting for the future," *Management & Sustainability: An Arab Review*, vol. 3, no. 1, pp. 16–29, 2024.
- [19] S. Sarsa, P. Denny, A. Hellas, and J. Leinonen, "Automatic generation of programming exercises and code explanations using large language models," in *Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 1*, 2022, pp. 27–43.
- [20] J. Leinonen, A. Hellas, S. Sarsa, *et al.*, "Using large language models to enhance programming error messages," in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, 2023, pp. 563–569.

- [21] S. MacNeil, A. Tran, D. Mogil, S. Bernstein, E. Ross, and Z. Huang, "Generating diverse code explanations using the gpt-3 large language model," in *Proceedings of the 2022 ACM conference on international computing education research-volume 2*, 2022, pp. 37-39.
- [22] J. Savelka, A. Agarwal, C. Bogart, and M. Sakr, "Large language models (gpt) struggle to answer multiple-choice questions about code," *arXiv preprint arXiv:2303.08033*, 2023.
- [23] M. J. Page, J. E. McKenzie, P. M. Bossuyt, *et al.*, "The prisma 2020 statement: An updated guideline for reporting systematic reviews," *bmj*, vol. 372, 2021.
- [24] N. R. Haddaway, M. J. Page, C. C. Pritchard, and L. A. McGuinness, "Prisma2020: An r package and shiny app for producing prisma 2020-compliant flow diagrams, with interactivity for optimised digital transparency and open synthesis," *Campbell systematic reviews*, vol. 18, no. 2, e1230, 2022.
- [25] G. Jošt, V. Taneski, and S. Karakatič, "The impact of large language models on programming education and student learning outcomes," *Applied Sciences*, vol. 14, no. 10, p. 4115, 2024.
- [26] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman, "Studying the effect of ai code generators on supporting novice learners in introductory programming," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023, pp. 1-23.
- [27] B. Sheese, M. Liffiton, J. Savelka, and P. Denny, "Patterns of student help-seeking when using a large language model-powered programming assistant," in *Proceedings of the 26th Australasian computing education conference*, 2024, pp. 49-57.
- [28] M. Q. Feldman and C. J. Anderson, "Non-expert programmers in the generative ai future," in *Proceedings of the 3rd Annual Meeting of the Symposium on Human-Computer Interaction for Work*, 2024, pp. 1-19.
- [29] J. Chen, X. Lu, Y. Du, *et al.*, "Learning agent-based modeling with llm companions: Experiences of novices and experts using chatgpt & netlogo chat," in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1-18.

- [30] S. Prasad, B. Greenman, T. Nelson, and S. Krishnamurthi, "Generating programs trivially: Student use of large language models," in *Proceedings of the ACM Conference on Global Computing Education Vol 1*, 2023, pp. 126–132.
- [31] Y. Xue, H. Chen, G. Bai, R. Tairas, and Y. Huang, "Does chatgpt help with introductory programming," in *An experiment of students using ChatGPT in CS1 The 46th International Conference on Software Engineering: Software Engineering Education and Training*, Lisbon, 2024.
- [32] T. Kosar, D. Ostojić, Y. D. Liu, and M. Mernik, *Computer science education in chatgpt era: Experiences from an experiment in a programming course for novice programmers. mathematics* 12, 5 (2024), 2024.
- [33] J. Prather, B. N. Reeves, P. Denny, *et al.*, "'it's weird that it knows what i want': Usability and interactions with copilot for novice programmers," *ACM transactions on computer-human interaction*, vol. 31, no. 1, pp. 1–31, 2023.
- [34] E. Frankford, C. Sauerwein, P. Bassner, S. Krusche, and R. Breu, "Ai-tutoring in software engineering education," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training*, 2024, pp. 309–319.
- [35] S. Nguyen, H. M. Babe, Y. Zi, A. Guha, C. J. Anderson, and M. Q. Feldman, "How beginning programmers and code llms (mis) read each other," in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–26.
- [36] R. Budhiraja, I. Joshi, J. S. Challa, H. D. Akolekar, and D. Kumar, "'it's not like jarvis, but it's pretty close!'-examining chatgpt's usage among undergraduate students in computer science," in *Proceedings of the 26th Australasian Computing Education Conference*, 2024, pp. 124–133.
- [37] M. Amoozadeh, D. Daniels, D. Nam, *et al.*, "Trust in generative ai among students: An exploratory study," in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, 2024, pp. 67–73.
- [38] J. Prather, P. Denny, J. Leinonen, *et al.*, "The robots are here: Navigating the generative ai revolution in computing education," in

- Proceedings of the 2023 working group reports on innovation and technology in computer science education*, 2023, pp. 108–159.
- [39] A. Husain, “Potentials of chatgpt in computer programming: Insights from programming instructors,” *Journal of Information Technology Education: Research*, vol. 23, p. 002, 2024.
 - [40] S. Lau and P. Guo, “From "ban it till we understand it" to "resistance is futile": How university programming instructors plan to adapt as more students use ai code generation and explanation tools such as chatgpt and github copilot,” in *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 1*, 2023, pp. 106–121.
 - [41] Z. Pan, Z. Xie, T. Liu, and T. Xia, “Exploring the key factors influencing college students’ willingness to use ai coding assistant tools: An expanded technology acceptance model,” *Systems*, vol. 12, no. 5, p. 176, 2024.
 - [42] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, “Using an llm to help with code understanding,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
 - [43] A. Neyem, L. A. González, M. Mendoza, J. P. S. Alcocer, L. Centellas, and C. Paredes, “Towards an ai knowledge assistant for context-aware learning experiences in software capstone project development,” *IEEE Transactions on Learning Technologies*, 2024.
 - [44] D. Jayagopal, J. Lubin, and S. E. Chasins, “Exploring the learnability of program synthesizers by novice programmers,” in *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, 2022, pp. 1–15.
 - [45] M. Liffiton, B. E. Sheese, J. Savelka, and P. Denny, “Codehelp: Using large language models with guardrails for scalable support in programming classes,” in *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*, 2023, pp. 1–11.
 - [46] M. Kazemitabaar, R. Ye, X. Wang, *et al.*, “Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs,” in *Proceedings of the 2024 chi conference on human factors in computing systems*, 2024, pp. 1–20.

- [47] S. Gumina, T. Dalton, and J. Gerdes, "Teaching it software fundamentals: Strategies and techniques for inclusion of large language models: Strategies and techniques for inclusion of large language models," in *Proceedings of the 24th Annual Conference on Information Technology Education*, 2023, pp. 60–65.
- [48] C. Bull and A. Kharrufa, "Generative artificial intelligence assistants in software development education: A vision for integrating generative artificial intelligence into educational practice, not instinctively defending against it," *IEEE Software*, vol. 41, no. 2, pp. 52–59, 2023.
- [49] J. Prather, J. Leinonen, N. Kiesler, *et al.*, "Beyond the hype: A comprehensive review of current trends in generative ai research, teaching practices, and tools," *2024 Working Group Reports on Innovation and Technology in Computer Science Education*, pp. 300–338, 2025.
- [50] P. Denny, V. Kumar, and N. Giacaman, "Conversing with copilot: Exploring prompt engineering for solving cs1 problems using natural language," in *Proceedings of the 54th ACM technical symposium on computer science education V. 1*, 2023, pp. 1136–1142.
- [51] W. Cain, "Prompting change: Exploring prompt engineering in large language model ai and its potential to transform education," *TechTrends*, vol. 68, no. 1, pp. 47–57, 2024.
- [52] F. Miao, W. Holmes, R. Huang, H. Zhang, *et al.*, *AI and education: A guidance for policymakers*. Unesco Publishing, 2021.
- [53] A. Pang and F. Vahid, "Chatgpt and cheat detection in cs1 using a program autograding system," in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, 2024, pp. 367–373.

A. Appendix

Questions in data extraction forms
Question 1.1) What are the years the paper was published?
Question 1.2) How many participants are in the proposed experiment?
Question 1.3) What set of Programming Languages were tested in the experiment?
Question 1.4) Where is the experiment conducted?
Question 1.5) What research methods were conducted in the paper?
Question 2) What model of LLM is tested in the paper?
Question 3) What are the report features of proposed LLM-based coding assistants?
Question 4) What type of queries do participants create and send to LLM-based coding assistants?
Question 5) What is the reported upside for the user of proposed LLM-based coding assistants?
Question 6) What is the reported downside for users of proposed LLM-based coding assistants?
Question 7) What is the report limitation of proposed LLM-based coding assistants?
Question 8) What are the challenges in using proposed LLM-based coding assistants?

Table 1.1. Questions in data extraction forms

Author	Title	Year
Budhiraja et al.	"It's not like Jarvis, but it's pretty close!"-Examining ChatGPT's Usage among Undergraduate Students in Computer Science	2024
Amoozadeh et al.	Trust in Generative AI among Students: An exploratory study	2024
Nam et al.	Using an LLM to help with code understanding	2024
Neyem et al.	Towards an AI knowledge assistant for context-aware learning experiences in software capstone project development	2024
Prather et al.	The robots are here: Navigating the generative AI revolution in computing education	2023
Jošt et al.	The impact of large language models on programming education and student learning outcomes	2024
Kazemitabaar et al.	Studying the effect of AI code generators on supporting novice learners in introductory programming	2023
Husain	Potentials of ChatGPT in computer programming: Insights from programming instructors	2024
Sheese et al.	Patterns of student help-seeking when using a large language model-powered programming assistant	2024
Feldman and Anderson	Non-expert programmers in the generative AI future	2024
Chen et al.	Learning agent-based modeling with LLM companions: Experiences of novices and experts using ChatGPT & NetLogo chat	2024
Prasad et al.	Generating programs trivially: student use of large language models	2023
Lau and Guo	From "Ban it till we understand it" to "Resistance is futile": How university programming instructors plan to adapt to AI tools	2023
Jayagopal et al.	Exploring the learnability of program synthesizers by novice programmers	2022
Pan et al.	Exploring the key factors influencing college students' willingness to use AI coding assistant tools	2024
Xue et al.	Does ChatGPT Help with Introductory Programming	2024
Kosar et al.	Computer Science Education in ChatGPT Era: Experiences from an Experiment in a Programming Course for Novice Programmers	2024
Liffiton et al.	CodeHelp: Using large language models with guardrails for scalable support in programming classes	2023
Prather et al.	"It's weird that it knows what I want": Usability and interactions with Copilot for novice programmers	2023
Frankford et al.	AI-tutoring in software engineering education	2024
Nguyen et al.	How beginning programmers and code LLMs (mis) read each other	2024
Kazemitabaar et al.	CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant that balances student and educator needs	2024

Table 1.2. Papers included in literature review